In [1]:
```
# a. Import necessary packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# b. Load training and testing data
train = pd.read_csv('fashion-mnist_train.csv')
test = pd.read_csv('fashion-mnist_test.csv')

# Separate images and labels
X_train = train.iloc[:, 1:].values
y_train = train.iloc[:, 0].values

X_test = test.iloc[:, 1:].values
y_test = test.iloc[:, 0].values

# Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# c. Define the network architecture
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile the model
model.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# d. Train the model for 11 epochs
history = model.fit(
    X_train, y_train,
    epochs=11,
    batch_size=32,
    validation_split=0.1
)

# e. Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")

# f. Plot training loss and accuracy
plt.figure(figsize=(12, 5))

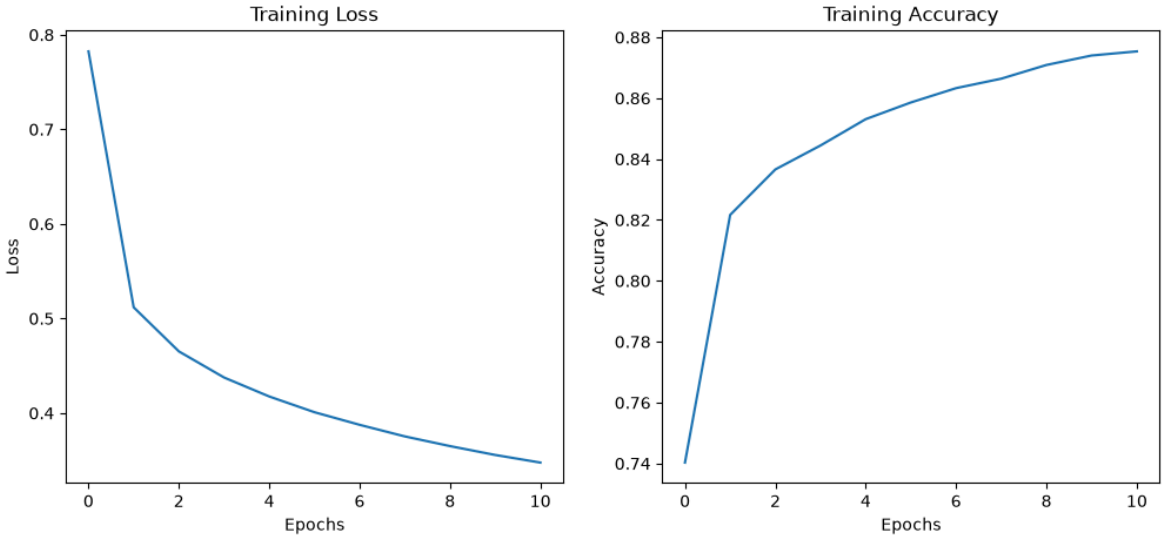
# Loss graph
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'])
plt.title('Training Loss')
```

```
plt.xlabel('Epochs')
plt.ylabel('Loss')

# Accuracy graph
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'])
plt.title('Training Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')

plt.show()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
C:\Users\manis\AppData\Local\Python\pythoncore-3.12-64\Lib\site-packages\keras\sr
c\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.7403 - loss: 0.7824 - va
l_accuracy: 0.7990 - val_loss: 0.5875
Epoch 2/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8216 - loss: 0.5118 - va
l_accuracy: 0.8195 - val_loss: 0.5123
Epoch 3/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8366 - loss: 0.4652 - va
l_accuracy: 0.8283 - val_loss: 0.4898
Epoch 4/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8445 - loss: 0.4376 - va
l_accuracy: 0.8430 - val_loss: 0.4626
Epoch 5/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8531 - loss: 0.4175 - va
l_accuracy: 0.8517 - val_loss: 0.4365
Epoch 6/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8586 - loss: 0.4009 - va
l_accuracy: 0.8463 - val_loss: 0.4386
Epoch 7/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8633 - loss: 0.3876 - va
l_accuracy: 0.8498 - val_loss: 0.4413
Epoch 8/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8664 - loss: 0.3753 - va
l_accuracy: 0.8537 - val_loss: 0.4235
Epoch 9/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8709 - loss: 0.3651 - va
l_accuracy: 0.8448 - val_loss: 0.4396
Epoch 10/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8740 - loss: 0.3557 - va
l_accuracy: 0.8463 - val_loss: 0.4512
Epoch 11/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8754 - loss: 0.3477 - va
l_accuracy: 0.8588 - val_loss: 0.4015
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8663 - loss: 0.3759
Test Loss: 0.3759397566318512
Test Accuracy: 86.62999868392944 %
```


Training Loss
Loss
Epochs
0.8
0.7
0.6
0.5
0.4
0
2
4
6
8
10
Training Accuracy
Accuracy
Epochs
0.88
0.86
0.84
0.82
0.80
0.78
0.76
0.74
0
2
4
6
8
10

In [ ]: